

Pizzo: A Gamified Route-Planning Application Combining the Chinese Postman Problem and Blackjack Decision Support

Tarik Agić

Faculty of Electrical Engineering, University of Sarajevo

Sarajevo, Bosnia and Herzegovina

Student ID: 20108

Course: Artificial Intelligence

Abstract—*Pizzo* is an interactive desktop application that combines a graph-optimization task with a probabilistic decision-making mini-game. In the narrative framing, “pizzo” refers to protection money collected by an Italian mafia capo. If a city is modeled as a weighted, connected, undirected graph where intersections are vertices and streets are edges, then Toni must traverse the city to visit every business located along the streets. To minimize fuel cost and exposure risk, Toni seeks the shortest closed walk that covers every street at least once—the *Chinese Postman Problem* (CPP) [1]. After a long day of collecting pizzo, Toni relaxes by playing Blackjack (“21”), trying to increase his bankroll. The Blackjack module includes an *Advisor* (“Consigliere”) that computes the expected value (EV) of Hit versus Stand under an infinite-deck probability model, and recommends the statistically best action.

Index Terms—Chinese Postman Problem, graph optimization, Dijkstra’s algorithm, minimum-weight perfect matching, Euler circuit, Blackjack, expectimax, expected value.

I. INTRODUCTION

This project implements a single application with two tightly connected gameplay loops: (1) a route-planning loop based on the Chinese Postman Problem (CPP) on an undirected weighted graph, and (2) a Blackjack mini-game that shares the same bankroll as the route-planning loop.

The CPP portion models “neighborhoods” as weighted undirected graphs loaded from plain text files. The user selects a neighborhood, chooses a start node by clicking a vertex, and the application computes and animates a CPP tour beginning at that vertex. The animation includes a speed slider and a start/stop control, and the traversal updates the global wallet according to a reward/penalty formula implemented in the code. The Blackjack portion is opened from the main window as a separate window; it provides a standard HIT/STAND flow with a dealer that hits until ≥ 17 , and it uses the same wallet and bet state as the main mode.

A key design goal was that every number shown in the GUI is computed from the underlying algorithmic state: for example, the main HUD shows the sum of unique edge

weights (original graph cost), the number of odd vertices, and the realized CPP tour cost (sum of step costs in the final walk). These values correspond directly to fields computed in `MainView.h` and the CPP solver modules.

II. BACKGROUND AND PROBLEM DEFINITIONS

A. City as a Graph

A neighborhood is modeled as an undirected connected graph $G = (V, E)$. Each vertex $v \in V$ represents an intersection (or a point of interest), and each edge $e = (u, v) \in E$ represents a street segment connecting two intersections. Each edge has a positive integer weight $w(u, v)$ representing traversal cost (distance).

In the implementation, the graph is stored as an adjacency list. When an undirected edge is added, two directed adjacency entries are inserted (one for each direction), but they share a single unique edge identifier. This shared identifier is crucial for Euler traversal because it allows each undirected edge to be marked “used” exactly once.

B. Euler Circuits and Odd Vertices

An *Euler circuit* is a closed walk that uses every edge exactly once. A connected undirected graph has an Euler circuit if and only if *every vertex has even degree* [2]. Vertices of odd degree are central to CPP: if any vertex has odd degree, then no Euler circuit exists without duplicating some edges. The CPP solution is therefore “Eulerization”: duplicating a set of edges (or paths) so that all degrees become even, enabling an Euler circuit in an augmented multigraph.

C. Chinese Postman Problem (CPP)

The undirected CPP asks for the minimum-cost closed walk that traverses every edge at least once [1]. For undirected graphs, a classical solution pipeline is:

- 1) Compute all odd-degree vertices.
- 2) Compute shortest-path distances between all pairs of odd vertices.
- 3) Find a minimum-weight perfect matching on the odd vertices using those distances.

- 4) Duplicate the shortest paths for the matched pairs (augment the graph).
- 5) Compute an Euler circuit in the augmented graph (e.g., Hierholzer’s algorithm) [3].

This project implements exactly that pipeline in `CPP.cpp` using `Dijkstra`, a brute-force minimum perfect matching routine, and a Hierholzer-style Euler circuit routine.

D. Blackjack (“21”)

Blackjack is a card game in which the player attempts to obtain a hand value as close to 21 as possible without exceeding it (busting). Cards 2–10 have their face value, J/Q/K count as 10, and Ace can be 11 or 1 depending on what keeps the hand legal. The player chooses HIT to draw another card or STAND to end drawing; then the dealer plays under deterministic rules (hit until ≥ 17 , stand otherwise), and the hands are compared.

In this project, Blackjack is simplified to the HIT/STAND action space with the dealer hitting until total ≥ 17 . The Advisor computes the expected value of actions under an infinite-deck rank distribution (each rank is sampled with probability $1/13$).

III. SYSTEM OVERVIEW AND ARCHITECTURE

A. Modules

The codebase is split into:

- **Graph & Algorithms (CPP):** Graph, Dijkstra, Matching, Euler, CPP, and Route.
- **Neighborhood I/O:** Loader and optional NeighborhoodPicker.
- **Blackjack Core:** Card, Hand, BlackjackGame, Advisor.
- **GUI:** Application, MainWindow/MainView, BlackjackWindow/BlackjackView, shared GameState, and CommonTypes.

B. Shared State Between Modes

The two modes share a global wallet and bet through `GameState.h`, implemented as `inline static` integers. The wallet is updated both by the CPP traversal (rewards/penalties) and by Blackjack payouts. This design makes the Blackjack mini-game meaningfully integrated rather than a separate demonstration.

IV. NEIGHBORHOOD DATA, LOADING, AND AUTOMATIC VISUALIZATION

A. Neighborhood File Format

Neighborhood graphs are loaded from plain text files using `Loader`. The format supported by the loader includes:

- `V n`: number of vertices.
- `LABELS ...`: optional vertex labels (strings).
- `E u v cost`: an undirected weighted edge.
- Comment lines beginning with `#` are ignored.

This format allows creating new neighborhoods without modifying the code: as long as a file describes V , optional labels, and edges, the loader will produce a graph, and the GUI will draw it.

TABLE I
NEIGHBORHOOD GRAPH STATISTICS (FROM `NEIGHBORHOOD1-3.TXT`)

Neighborhood	$ V $	$ E $	$\sum w(e)$	#odd
Little Italy	6	8	32	4
North Jersey	10	19	130	4
Pine Barrens	8	8	40	4

B. Provided Neighborhoods and Intended Variety

The GUI exposes three neighborhood choices defined in `MainView.h`: `LITTLE ITALY` (`neighborhood1.txt`), `NORTH JERSEY` (`neighborhood2.txt`), and `PINE BARRENS` (`neighborhood3.txt`). These three maps provide algorithmic variety:

- **Little Italy** is small (6 vertices, 8 edges) and is useful for quickly visualizing the CPP pipeline.
- **North Jersey** is larger and denser (10 vertices, 19 edges), creating more route options and a more complex Eulerization effect.
- **Pine Barrens** is relatively sparse (8 vertices, 8 edges), illustrating how CPP handles lower connectivity while still requiring Eulerization when odd vertices exist.

Table I summarizes the exact counts computed from the provided neighborhood text files.

C. Automatic Node Layout in the GUI

An automatic node-layout routine is implemented directly in the GUI. After a neighborhood is loaded, the GUI computes positions for all vertices by placing them evenly on a circle: for each vertex index $i \in \{0, \dots, N-1\}$, an angle

$$\theta_i = \frac{2\pi i}{N}$$

is used, and the node is placed at

$$(x_i, y_i) = (x_c + R \cos \theta_i, y_c - R \sin \theta_i),$$

where (x_c, y_c) is the graph-rectangle center and R is a radius chosen from the available panel size. This matches the implemented loop in `MainView.h` which computes `ang`, `cos`, `sin`, and rounds the results to pixel coordinates.

Therefore, to add a new neighborhood, one only needs to add a new text file following the loader format and add a corresponding button entry; the drawing is automatic because it depends only on N and the view rectangle.

V. CHINESE POSTMAN PROBLEM: ALGORITHM AND IMPLEMENTATION

A. Pipeline Implemented in `CPP.cpp`

The core CPP routine follows the classical undirected CPP method. At a high level:

The remainder of this section ties each pipeline step to a specific module and clarifies how cost is computed.

Algorithm 1 Undirected Chinese Postman as Implemented

Require: Connected undirected weighted graph $G = (V, E)$, start vertex s

Ensure: Minimum-cost closed walk starting at s that covers all edges

```
1:  $O \leftarrow \{v \in V \mid \deg(v) \text{ is odd}\}$ 
2: if  $O = \emptyset$  then
3:   return EulerCircuit( $G, s$ )
4: end if
5: For each  $o_i \in O$ : run Dijkstra( $G, o_i$ ) to get distances to all  $o_j \in O$ 
6: Build distance matrix  $D[i, j] = \text{dist}(o_i, o_j)$ 
7:  $M \leftarrow \text{MinPerfectMatching}(O, D)$ 
8:  $G' \leftarrow G$   $\triangleright$  copy as multigraph
9: for all pairs  $(o_i, o_j) \in M$  do
10:    $P \leftarrow$  reconstruct shortest vertex path from  $o_i$  to  $o_j$ 
11:   duplicate each edge along  $P$  into  $G'$ 
12: end for
13:  $C \leftarrow \text{EulerCircuit}(G', s)$ 
14: Convert consecutive vertices of  $C$  into route steps and return
```

B. Graph Representation and Edge Identifiers

Graph stores adjacency lists, and each undirected edge is represented by two directed adjacency entries that share the same edge ID. Euler traversal uses a boolean array `used[edgeId]` so an undirected edge is consumed exactly once, even though it appears twice in adjacency.

This is essential for Hierholzer traversal in undirected graphs: when the algorithm selects one directed adjacency entry, it marks the underlying undirected edge as used and will skip the opposite direction later.

C. Odd Vertices and Why They Matter

If the graph is not Eulerian, the number of odd-degree vertices is even. The CPP solution adds (duplicates) edges so that odd vertices become even, enabling an Euler circuit. In the GUI, the number of odd vertices is computed from the loaded graph and displayed in the HUD (e.g., “Odd Vertices: 4”).

D. Shortest Paths Between Odd Vertices (Dijkstra)

To decide which edges to duplicate, the solver requires the shortest-path distances between odd vertices. The project implements Dijkstra’s algorithm over the adjacency list. For each odd vertex o_i , Dijkstra computes distances to all vertices and stores a parent pointer array, which later enables reconstruction of an explicit path between matched vertices.

E. Minimum Perfect Matching of Odd Vertices

The solver forms pairs of odd vertices such that the sum of distances between paired vertices is minimized.

This is a minimum-weight perfect matching problem on the complete graph induced by odd vertices, with edge weights equal to shortest-path distances.

In this codebase, `Matching.cpp` implements a brute-force recursive pairing strategy: it picks the first unpaired odd vertex, tries pairing it with every other unpaired odd vertex, and recurses to complete the matching, keeping the minimal total cost. This approach is exact for small numbers of odd vertices and matches the intended scale of the provided neighborhoods (each has exactly four odd vertices, i.e., only three possible matchings).

F. Eulerization by Duplicating Shortest Paths

For each pair (o_i, o_j) in the matching, the solver reconstructs the shortest vertex path in the original graph and duplicates each edge along that path into an augmented graph G' . Because the augmented graph is a multigraph, duplicating edges increases degrees appropriately without deleting original edges.

G. Euler Circuit via Hierholzer-Style Traversal

Once G' is Eulerian (all degrees even), the solver computes an Euler circuit beginning at the selected start node. The implementation in `Euler.cpp` is a Hierholzer-style algorithm and includes a correctness check that the circuit length is $|E'| + 1$; otherwise, it throws an error indicating a disconnected graph (no valid Euler circuit).

H. Cost Accounting in the HUD

The main view displays three key quantities:

- **Sum of Edges:** the sum of weights of all original edges counted once (unique edge cost).
- **Odd Vertices:** the number of odd-degree vertices in the original graph.
- **Real Path Cost:** the total cost of the computed CPP route, defined as the sum of costs of the step sequence produced from the Euler circuit in the augmented graph.

The CPP solver explicitly returns:

- **originalCost:** the original unique edge sum.
- **addedCost:** the extra cost introduced by duplicating shortest paths for the matching.
- **totalCost:** `originalCost + addedCost`.

The GUI additionally computes the realized route cost from the produced step list.

I. Representative Code Excerpt (CPP orchestration)

The following excerpt illustrates that the solver (i) finds odd vertices, (ii) runs Dijkstra among odd vertices, (iii) matches them, (iv) augments the graph, and (v) computes Euler:

```
auto odd = findOddVertices(g);
if (odd.empty()) {
    res.eulerCircuit = Euler::eulerCircuit(g, start);
    return res;
}
auto dist = buildOddDistanceMatrix(g, odd); // via Dijkstra
```

```

auto mr = Matching::minPerfectMatching(odd, dist);
Graph aug = g;
augmentByMatchedPaths(aug, g, odd, mr.pairs); // duplicate
        shortest paths
res.eulerCircuit = Euler::eulerCircuit(aug, start);

```

(Names are simplified for readability; this excerpt corresponds to the concrete control flow in `CPP.cpp`.)

VI. GAMIFICATION: MONEY COLLECTION LOGIC DURING TRAVERSAL

A. Global Wallet

The wallet is stored in `GameState::Wallet` and is shared across both game modes. The CPP traversal updates the wallet each time the animated agent completes a route step.

B. Per-Edge Reward and Per-Step Penalty

The money logic is implemented in `MainView.h` with three constants: `BASE_REWARD = 40`, `WEIGHT_BONUS = 10`, and `PENALTY = 3`. Let w be the integer weight of the traversed edge. Then:

- The first time an undirected edge is traversed, the wallet increases by

$$\Delta^+ = 40 + 10 \cdot w.$$

- Every traversal (including the first) decreases the wallet by

$$\Delta^- = 3 \cdot w.$$

Therefore, the *net* change on first traversal of an edge is $(40 + 10w) - 3w = 40 + 7w$, while revisiting an edge yields a net change of $-3w$.

To ensure “first time” is defined per *undirected* edge, the GUI tracks a set of rewarded edges keyed by the unordered pair $\{u, v\}$. This design reinforces the CPP objective: duplicated edges (required when odd vertices exist) impose additional penalties during traversal.

C. Traversal Speed Control

The traversal is driven by a GUI timer constructed with a period of `0.04f` seconds. Each tick advances the interpolation parameter by an animation speed derived from a slider. The speed is mapped linearly between `_animSpeedMin = 0.02f` and `_animSpeedMax = 0.18f`. Thus the GUI provides fine-grained control over how quickly the agent moves along route steps.

VII. BLACKJACK MODULE: RULES AND IMPLEMENTATION

A. Card and Hand Representation

Cards are represented by rank and suit. For game logic, the hand value is computed from ranks: 2–9 map to their numeric values, 10/J/Q/K map to 10, and Ace initially maps to 11. The `Hand` class tracks a running total and a boolean `soft` indicator. When adding a card would cause the total to exceed 21 and `soft` is true, the total is reduced by 10 and `soft` is set false, effectively converting one Ace from 11 to 1.

B. Round Flow

The Blackjack game state:

- Deals two cards to the player and two cards to the dealer at round start.
- Exposes two actions: HIT (draw one card) and STAND (end drawing).
- Implements the dealer policy: hit while dealer total < 17 ; otherwise stand.

C. Betting and Payout

The GUI uses `GameState::CurrentBet` and clamps it to the wallet. At round start, the bet amount is subtracted from the wallet. At round end, the payout is applied once:

- Win (player wins or dealer busts): add $2 \times$ bet (restoring stake plus winnings).
- Tie: add $1 \times$ bet (stake returned).
- Loss: no addition (stake already removed).

VIII. ADVISOR (CONSIGLIERE): EXPECTED-VALUE DECISION SUPPORT

A. Probability Model

The Advisor assumes an infinite deck with uniform rank probability. Each of the 13 ranks is drawn with probability $1/13$; consequently, the total probability of drawing a 10-valued card is $4/13$ (Ten, Jack, Queen, King).

B. Dealer Outcome Distribution

Given a dealer upcard rank, the Advisor computes a distribution over dealer terminal outcomes (bust or totals 17–21) by enumerating all possible hole-card ranks and recursively simulating the dealer policy (hit until ≥ 17). This recursion is memoized over dealer states (`total`, `soft`).

C. Player Expected Value

Given the player’s current state (`total`, `soft`) and the dealer upcard, the Advisor computes:

- EV(STAND): compare player total against the dealer distribution and assign payoff +1 for win, 0 for tie, -1 for loss.
- EV(HIT): enumerate each next card rank with probability $1/13$, update the player’s state, and then evaluate the best achievable EV from that new state.

The continuation value after a hit is computed as:

$$EV(s) = \max\{EV(\text{HIT} \mid s), EV(\text{STAND} \mid s)\},$$

which is an expectimax-style optimal policy evaluation for a stochastic environment.

The Advisor memoizes player states by (`total`, `soft`, `dealerUp`) to ensure responsiveness in the GUI. Finally, it recommends HIT if $EV(\text{HIT}) > EV(\text{STAND})$ and STAND otherwise, and displays both EV values in the advisor tooltip.

IX. GRAPHICAL USER INTERFACE AND FEATURES

A. Main Window (CPP Mode)

The main window, titled **Pizzo**, contains:

- A HUD with a play/pause button, traversal speed slider, and money display.
- A graph panel that renders the neighborhood graph and the computed tour.
- A neighborhood-selection panel with three provided neighborhood options.
- A **Blackjack** button that opens the Blackjack window.

The user selects a neighborhood, then clicks a node to begin; the HUD text indicates this state (“Select a node to begin”).

B. Blackjack Window

The Blackjack window renders:

- The dealer hand (including a face-down card during play) and the player hand.
- HIT and STAND action buttons.
- A bankroll and bet control panel (increment/decrement buttons).
- A back button to close the window and return to the main mode.

X. CONCLUSION

Pizzo demonstrates an end-to-end application that connects deterministic graph optimization (CPP) with stochastic decision-making (Blackjack). The CPP mode implements the classical undirected CPP pipeline using Dijkstra shortest paths, minimum perfect matching of odd vertices, graph Eulerization, and a Hierholzer-style Euler circuit routine. The Blackjack mode provides a playable mini-game that shares a wallet with the CPP traversal and includes a mathematically grounded EV-based Advisor. Together, the system illustrates core algorithmic concepts (odd vertices, Euler circuits, shortest paths, matchings) and AI concepts (expectimax-style evaluation under chance) within a cohesive interactive narrative.

REFERENCES

- [1] MIT, *Urban Operations Research*, “Section 6.4.2.” [Online]. Available: https://web.mit.edu/urban_or_book/www/book/chapter6/6.4.2.html. Accessed: 2026-02-16.
- [2] Wolfram MathWorld, “Eulerian cycle.” [Online]. Available: <https://mathworld.wolfram.com/EulerianCycle.html>. Accessed: 2026-02-16.
- [3] TUM Algorithms, “Hierholzer’s algorithm.” [Online]. Available: https://algorithms.discrete.ma.tum.de/graph-algorithms/hierholzer/index_en.html#. Accessed: 2026-02-16.

APPENDIX

Figures 1 and 2 show the two main application modes.

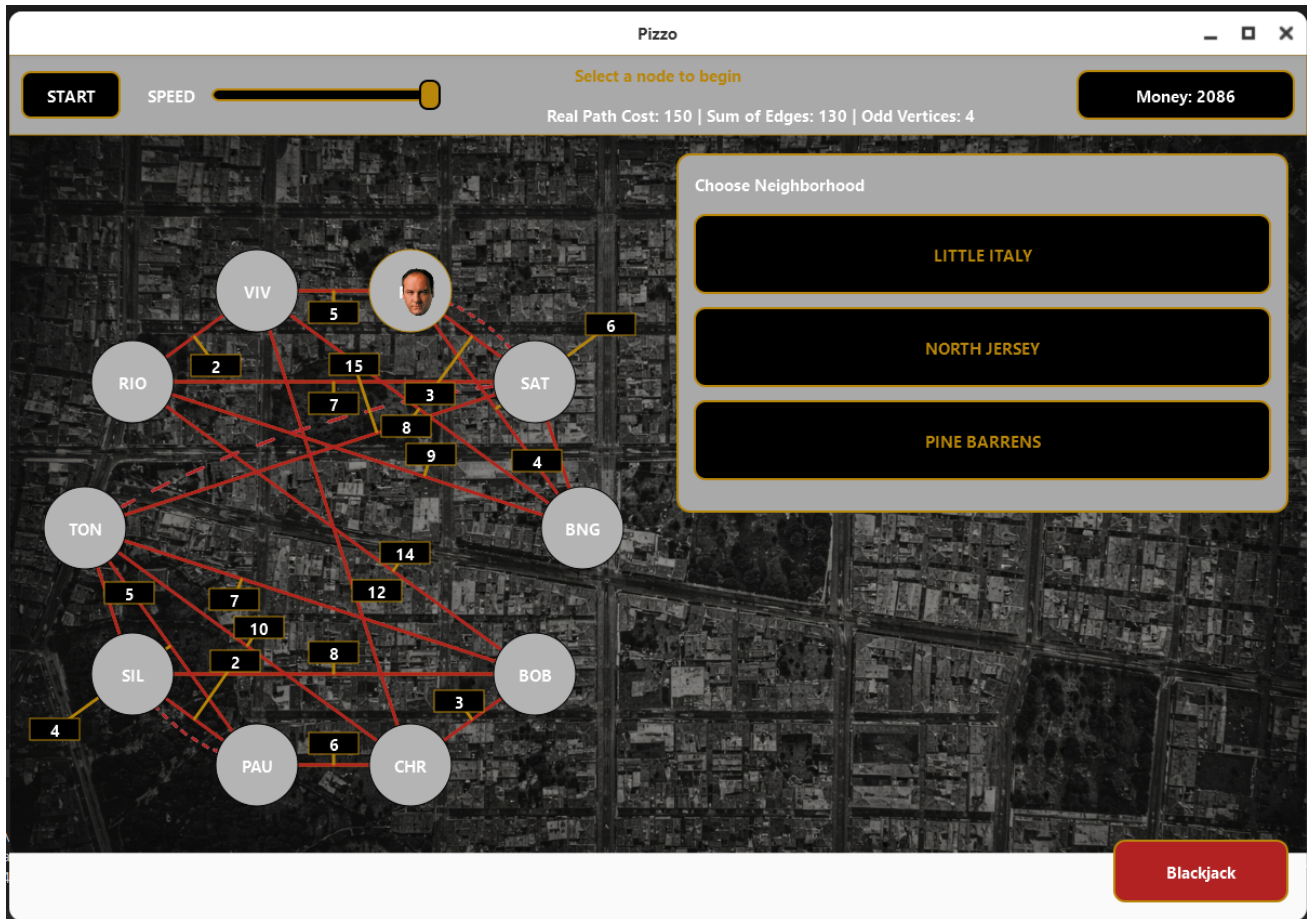


Fig. 1. Main mode (CPP): neighborhood selection, graph visualization, route statistics, speed control, and shared money display.



Fig. 2. Blackjack mode: dealer and player hands, HIT/STAND actions, bankroll and bet controls, and integration with shared wallet.